Assignment 3: Prompt Tuning

Course: Foundation for Large Language Models (FLLM - AI60213)

Due Date: November-8-2025

[Total marks: 20]

Advisory

Using GenAI tools for solving assignments is discouraged, as this will increase the probability of plagiarism with others who may choose the same path.

Submission Instructions

1. Only one member from each group needs to submit the assignment through Moodle to avoid any duplicate submissions.
2. Submit the solution file as: `Group(number)_assignment3_solution.ipynb` and also include a corresponding `.py` file.

Plagiarism Policy

1. Code plagiarism will be checked on the designated blocks which the students have to code.
2. No penalty till 30% (Baseline).
3. Each 10% increase over baseline will cost 1 mark. Beyond 70%, absolute -2 (negative).
4. Two groups having similar code will be penalized with the same amount. No arbitration will be done.
5. Late submission policy: One week window with a penalty of 3 marks.

General Instructions

This assignment is designed to give you a hands-on understanding of Prompt Tuning in LLMs. You are required to do experiments using **BART-base** [1] model and prompt-tuning for **FIVE tasks** (COPA, etc.) from SuperGLUE [3].

You should submit a single PDF report containing your answers to the questions, including any plots and tables. Your report should clearly explain your methodology,

observations, and conclusions for each question. You should also submit your source code (e.g., a Jupyter Notebook file) with clear comments.

For your experiments, you can use the Hugging Face 'transformers' and 'datasets' libraries. The '**facebook/bart-base**' model and '**aps/super_glue**' datasets from Huggingface should be sufficient for all tasks and can be run in a Google Colab environment.

(Note: Please note that all the hyperparameters are already defined in the provided code file. Kindly use those parameters while executing the assignment. If you wish to experiment with different hyperparameters for higher performance, you may do so — but make sure to clearly mention the changes and provide justification for each modification in your submission report.)

Question 1: XPROMPT-inspired pruning for efficient soft prompt tuning.

[Marks: 20 (16 + 2 + 2)]

Background: Prompt tuning is a parameter-efficient alternative to fine-tuning large pre-trained language models (PLMs) or large language models (LLMs). However, as shown in the paper [2], not all prompt tokens contribute equally to model performance - some may even degrade it. XPROMPT [2] leverages the *lottery ticket hypothesis* to identify and eliminate such "negative" prompt tokens through hierarchical structured pruning, thus obtaining a minimal yet effective set of prompt parameters. This question requires you to implement a simplified XPROMPT framework by introducing pruning mechanisms on soft prompts trained for downstream text classification, FIVE tasks from SuperGLUE [3] - COPA, WiC, WSC, CB, and RTE.

Tasks:

1. Implement the above XPROMPT-inspired pruning (As described in Box below) framework using `Hugging Face Transformers`. Use the `BART-base` model and classification datasets (e.g., COPA) from SuperGLUE [3]. Visualize token importance scores (e.g., bar plot) and show which tokens were pruned.

2. After pruning and retraining, evaluate the model performance (accuracy, loss) and report the reduction in tunable parameters compared to vanilla soft prompt tuning.

3. Discuss your observations on how pruning affects the model's efficiency and performance. Does XPROMPT achieve a better trade-off compared to full soft prompt tuning?

XPROMPT-inspired Soft Prompt Pruning Framework

Input: a pre-trained BART-base model (`base`), a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$, prompt length m, and pruning ratio r.
Steps:

1. **Soft Prompt Tuning (XPROMPT[2] Section -4.1):** Prepend m learnable soft prompt tokens to the input sequence embeddings of BART-base. Freeze all model parameters except these soft prompt embeddings and fine-tune on $\mathcal{D}$.

2. **Token-level Pruning (XPROMPT[2] Section -4.2.1):** After training, compute the importance score of each soft prompt token using:

$$I_{p_i} = \mathbb{E}_{x \sim \mathcal{D}} \left| \frac{\partial L(x)}{\partial p_i} \right|$$

   Rank tokens by importance and prune the lowest $r\%$ (i.e., set them to zero or remove them).

3. **Piece-level Pruning (XPROMPT[2] Section -4.2.2):** Divide each remaining prompt embedding vector into k equal parts $\{q_1, q_2, \ldots, q_k\}$ and compute piece-level importance scores:

$$I_{q_j} = \mathbb{E}_{x \sim \mathcal{D}} \left| \frac{\partial L(x)}{\partial q_j} \right|$$

   Prune low-importance pieces within each token.

4. **Rewinding (XPROMPT[2] Section -4.3):** Reinitialize the remaining (unpruned) prompt tokens using their pre-pruned values, retrain them with frozen BART-base parameters, and evaluate the model.

5. **Evaluation (XPROMPT[2] Section -5.3):** Compare model performance and number of tunable parameters before and after pruning.

Output: A fine-tuned BART-base model with reduced prompt parameters, its validation accuracy, and a comparison table summarizing compression and performance metrics.

Table 1: Summary of XPROMPT hyperparameters on SuperGLUE.

Hyperparameter	**Value / Description**
Prompt Length	20 soft tokens
Embedding Dim (d_{model})	1024
Piece Splits	16 per token
Optimizer	Adafactor
Learning Rate	0.3
Weight Decay	1×10^{-5}
Epochs	90 (tuning) + 10 (retrain per prune ratio)
Batch Size	16
Max Input Length	256 tokens
Max Output Length	8 tokens
Pruning Rates	[0.1, 0.3]

References

[1] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*, 2019.

[2] Fang Ma, Chen Zhang, Lei Ren, Jingang Wang, Qifan Wang, Wei Wu, Xiaojun Quan, and Dawei Song. XPrompt: Exploring the extreme of prompt tuning. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 11033–11047, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.758. URL `https://aclanthology.org/2022.emnlp-main.758/`.

[3] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems. *Advances in neural information processing systems*, 32, 2019.